

Apprentissage incrémental EWC sur microcontrôleur : parité FP32 mesurée, effondrement de la quantification naïve et récupération par kernel INT8 calibré

Léonard Rivals

ISAE-SUPAERO (DISC) — ENAC (LII) — Edge Spectrum

9 septembre 2026

Résumé

Nous étudions le portage d'un classifieur *Elastic Weight Consolidation* (EWC) pour la maintenance prédictive sur un microcontrôleur Cortex-M4 (NUCLEO-F439ZI, 256 kB SRAM, sans NPU). Trois résultats sont établis dans un protocole **apparié** (même graine, même ordre d'échantillons, même normalisation) entre l'exécution PC et la carte. **(1)** En FP32, la parité PC↔carte est *exacte* en mode gelé (1.000) et quasi exacte en mode en ligne (0.963–0.989), pour une latence inférence + mise à jour continue $\ll 100$ ms (Gap 2). **(2)** La quantification post-entraînement (PTQ) *naïve* de la tête EWC binaire s'effondre sur la carte : le F1 chute de ≈ 0.92 à 0.138 et 0.1337. **(3)** Une ablation bit-exacte émulée sur PC isole la cause (accumulateur `int8`, échelle par-tenseur non calibrée) et démontre la *récupération* par un kernel calibré : le schéma `per_tensor_calib` restaure +0.88 de F1 (Pronostia 0.9462, Monitoring 0.9201), et le schéma `q15` égale le FP32. **(4)** Cette récupération est désormais *mesurée sur carte* sur quatre cellules per-canal (F1 0.9173 Monitoring, 0.8995 Pronostia ; parité gelée 1.000 contre l'émulateur ; aucune erreur CRC). Nous étendons enfin l'étude au *moment* de la quantification, à la *profondeur* sub-INT8 et au budget système complet : la RAM des poids est divisée par 4, mais la RAM totale est inchangée ; la latence *augmente* (74 μ s contre 48–50 μ s, la requantification pesant un tiers du budget) et l'énergie ne bouge pas (67.47 μ J contre 67.65 μ J). Nous distinguons systématiquement les mesures *sur carte* des émulations *PC bit-exactes*, et laissons explicitement « à mesurer » les cellules non encore streamées.

Mots-clés : apprentissage continu ; EWC ; quantification INT8 ; PTQ ; microcontrôleur ; maintenance prédictive ; TinyML.

1 Introduction

La maintenance prédictive industrielle exige de plus en plus des modèles capables d'*apprendre en continu* sur l'équipement lui-même, sans retour vers un serveur : les conditions d'opération dérivent (usure, changement de régime, de machine), et un modèle figé se dégrade. L'apprentissage continu, ou Continual Learning (CL), répond à ce besoin [9, 3], mais son déploiement sur un Microcontroller Unit (MCU) reste contraint : mémoire vive de l'ordre de la centaine de kilo-octets, absence d'accélérateur neuronal, budget de latence strict.

Face à ces contraintes, la quantification INT8 est le réflexe le mieux établi de la littérature embarquée [13, 2]. Elle est presque toujours justifiée par un argument tripartite — moins de mémoire, moins de temps, moins d'énergie — rarement vérifié *sur la cible* pour chacun des trois termes séparément. C'est précisément ce que nous mesurons ici, et les trois termes ne se comportent pas de la même manière.

Le présent travail se positionne sur un **triple gap** que peu de travaux comblent simultanément : (Gap 1) validation sur des séries temporelles industrielles réelles ; (Gap 2) fonctionnement sous un budget mémoire et de latence *strict et mesuré* — ici 256 kB de Static Random Access Memory (SRAM) et 100 ms par cycle inférence + mise à jour ; (Gap 3) quantification INT8 compatible avec l'entraînement incrémental. Nous instancions ces trois axes sur un classifieur Elastic Weight Consolidation (EWC) [5] porté en C sur une carte NUCLEO-F439ZI (Cortex-M4 180 MHz, 256 kB SRAM, sans Neural Processing Unit (NPU)), évalué sur deux jeux industriels aux scénarios continus contrastés.

Contributions. (i) Un protocole *apparié* PC $\leftrightarrow$ carte (même graine, même ordre d'échantillons, même normalisation) établissant une parité FP32 *exacte* en mode gelé (1.000), quasi exacte en mode en ligne (de 0.9626 à 0.9887 sur les trois cellules), pour une latence inférence + apprentissage $\ll$ 100 ms. (ii) La mise en évidence, *sur carte réelle*, de l'effondrement de la Post-Training Quantization (PTQ) naïve de la tête EWC binaire : le F1 tombe de ≈ 0.92 à 0.138 et 0.1337 alors même que la Random Access Memory (RAM) des poids est bien divisée par 4. (iii) L'isolation de la cause par ablation *émulée bit-exacte*, puis la *récupération mesurée sur carte* avec un kernel à échelle calibrée : le F1 revient à 0.9173 et 0.8995, avec une parité gelée de 1.000 contre l'émulateur et aucune erreur Cyclic Redundancy Check (CRC). (iv) Deux axes complémentaires de quantification : le *moment*, où un entraînement Quantization-Aware Training (QAT) n'ajoute pas de gain décisif face à une PTQ correctement calibrée, et la *profondeur*, où la frontière dépend de la métrique retenue — ternaire en AUROC, INT4 en F1 mesuré sur carte — et où le gain mémoire n'existe qu'avec un véritable bit-packing. (v) Un budget système complet mesuré sur la même carte, qui *dissocie les trois promesses*

de l'INT8 : la RAM des poids est bien divisée par 4, mais la RAM totale du système est inchangée, la latence augmente d'environ la moitié et l'énergie ne bouge pas.

Ce dernier point est le fil conducteur de l'article. Sur cette cible, **l'INT8 se justifie par la mémoire, ni par la vitesse ni par l'énergie** — et encore, par une mémoire dont il faut préciser laquelle. Nous distinguons systématiquement, dans le texte comme dans les légendes, ce qui est *mesuré sur carte* de ce qui est *émulé bit-à-bit sur PC*, et laissons explicitement « à mesurer » toute cellule non encore streamée, plutôt que de l'estimer.

Organisation. La Section 2 situe le travail, la Section 3 décrit le modèle, le protocole apparié et les schémas de quantification, la Section 4 le dispositif. Les résultats se lisent en trois temps : parité, effondrement et récupération (Section 5), moment et profondeur (Section 6), budget système (Section 7). La Section 8 discute les deux paradoxes et les limites.

2 Travaux connexes

Apprentissage continu embarqué. Les méthodes de CL se répartissent classiquement en approches par régularisation, par rejeu et par architecture [9]. EWC [5] appartient à la première famille : une pénalité quadratique pondérée par l'information de Fisher protège les paramètres importants pour les tâches passées, sans stocker de données — une propriété précieuse sous contrainte mémoire. Sur MCU, Tiny Online Learning (TinyOL) [14] a montré la faisabilité de l'apprentissage en ligne d'une couche additionnelle, et Hyperdimensional Computing (HDC) [1] celle d'un apprentissage non neuronal à très basse consommation. LifeLearner [7] et les plateformes à rejeu latent quantifié [13] adressent le compromis mémoire/précision sur des cibles embarquées. Les panoramas Tiny Machine Learning (TinyML) récents [2] soulignent que l'entraînement *sur l'appareil* reste sous-exploré comparé à l'inférence seule, et les travaux appliqués à la maintenance prédictive [3] restent majoritairement évalués hors cible.

Quantifier : trois degrés de liberté, rarement croisés. On résume souvent la quantification à un choix de *format* — FP32 vers INT8, ou virgule fixe Q15. Deux autres degrés la commandent pourtant tout autant. Le *moment* d'abord [4, 6] : quantifier après l'entraînement, en PTQ, est immédiat mais sensible à la calibration de l'échelle, tandis que quantifier pendant, en QAT, coûte un réentraînement et suppose un accès au pipeline d'apprentissage, ce dont on ne dispose pas toujours en contexte industriel. La *profondeur* ensuite, avec la granularité qui l'accompagne : descendre sous 8 bits ne vaut que si l'échelle est estimée assez finement, par canal plutôt que par tenseur, et que si le stockage suit réellement le format annoncé. Ces trois

axes sont le plus souvent étudiés séparément ; nous les faisons varier sur *une même tête, un même jeu et une même carte*, ce qui permet de les comparer plutôt que de les juxtaposer.

Ce qui est rarement vérifié : les trois promesses, séparément. La quantification est habituellement justifiée par un argument tripartite — moins de mémoire, moins de temps, moins d’énergie — hérité de cibles disposant d’unités entières dédiées. Sur un cœur généraliste doté d’une Floating-Point Unit (FPU), rien ne garantit que les trois termes se comportent de la même façon, et les rapporter conjointement suppose une instrumentation que peu de travaux mettent en place : mesurer la mémoire *totale* du système et non la seule empreinte des poids, chronométrer le noyau poste par poste, et intégrer un courant réel à la sonde. Notre contribution empirique est de traiter ces trois promesses comme trois questions distinctes, mesurées sur la même carte : nous montrons que la première est tenue mais porte sur un poste minoritaire, et que les deux autres ne le sont pas. Nous établissons au passage l’effondrement de la PTQ naïve d’une tête de décision, en isolons la cause plutôt que de la masquer par un QAT en amont, et en démontrons la récupération par un noyau calibré.

3 Méthode

3.1 Formulation EWC et adaptation embarquée

Pénalité élastique. EWC combat l’oubli catastrophique en ancrant les paramètres jugés importants pour ce qui a déjà été appris [5]. Après une tâche, on retient un point d’ancrage $\theta^\star$ — les paramètres tels qu’ils étaient à la fin de cette tâche — et une estimation F de l’importance de chaque coordonnée. L’entraînement de la tâche suivante minimise alors la perte de tâche augmentée d’un rappel quadratique vers cet ancrage :

$$\mathcal{L}(\theta) = \mathcal{L}_{\text{t\^ache}}(\theta) + \frac{\lambda}{2} \sum_i F_i (\theta_i - \theta_i^\star)^2. \quad (1)$$

Le coefficient λ arbitre entre plasticité et rétention. Chaque coordonnée est rappelée avec une raideur qui lui est propre, λF_i : une coordonnée d’importance élevée devient coûteuse à déplacer, une coordonnée d’importance faible reste libre. La somme porte sur *tous* les paramètres de la tête, poids et biais compris. Il n’y a qu’un seul terme quadratique, ancré sur le dernier $\theta^\star$, et non une somme sur les tâches passées : le coût mémoire de la régularisation est donc constant, deux copies des paramètres, quel que soit le nombre de tâches déjà vues. Sur la première tâche, F et $\theta^\star$ n’existent pas encore et la pénalité est absente.

Estimation de l'importance sur PC. F est la diagonale de la matrice d'information de Fisher, estimée en fin de tâche sur les données de cette tâche. Nous en retenons la forme empirique, c'est-à-dire évaluée sur les étiquettes réelles et non sur des étiquettes tirées sous le modèle. La grandeur effectivement accumulée est le carré du gradient de la perte *moyennée sur un lot*, moyenné sur les B lots parcourus :

$$F_i = \frac{1}{B} \sum_{b=1}^B \left(\frac{\partial \mathcal{L}_b}{\partial \theta_i} \right)^2, \quad \mathcal{L}_b = \frac{1}{|b|} \sum_{n \in b} \ell(x_n, y_n; \theta). \quad (2)$$

Le budget est borné à 200 échantillons, ce qui suffit à ordonner les coordonnées entre elles — seul cet ordre compte, puisque λ absorbe l'échelle globale. Deux écritures de l'accumulation coexistent dans le projet. La variante *en ligne* au sens de [15] entretient une importance unique en la faisant décroître, $F \leftarrow \gamma F + F^{(t)}$ avec $\gamma = 0.9$, ce qui estompe progressivement les tâches les plus anciennes. La tête effectivement portée sur carte, elle, *remplace* son importance à chaque consolidation : l'ancrage suit toujours la dernière tâche consolidée.

Ce que la carte peut faire, et ce qu'elle ne peut pas. Le portage conserve la pénalité d'(1), dont la dérivée $\lambda F_i(\theta_i - \theta_i^*)$ s'ajoute au gradient de tâche. La mise à jour embarquée s'écrit, pour un poids W de la couche l :

$$W_{ji} \leftarrow W_{ji} - \eta \left(\underbrace{\delta_j a_i}_{\text{gradient de tâche}} + \underbrace{\lambda F_{ji} (W_{ji} - W_{ji}^*)}_{\text{rappel élastique}} \right), \quad \eta = 0.01. \quad (3)$$

La consolidation, en revanche, ne peut pas reproduire (2) : l'estimer demanderait de conserver un lot d'échantillons étiquetés et d'y repasser en rétropropagation, deux choses qu'une carte en flux continu, sans mémoire de données, n'a pas à sa disposition. L'importance y est donc approximée par la *magnitude* du poids lui-même, entretenue en moyenne mobile exponentielle :

$$F_{ji} \leftarrow \alpha F_{ji} + (1 - \alpha) W_{ji}^2, \quad W_{ji}^* \leftarrow W_{ji}, \quad \alpha = 0.99. \quad (4)$$

Il faut nommer cette substitution pour ce qu'elle est : W^2 n'est pas le Fisher, c'est un substitut d'importance qui n'exige ni donnée, ni étiquette, ni passe arrière, et dont le coût est une lecture-écriture par poids. Il fait l'hypothèse que les coordonnées de grande magnitude sont celles qui portent la fonction apprise. Deux conséquences suivent. La pénalité embarquée ne couvre que les matrices de poids, les biais n'ayant ni importance ni ancrage stockés, ce qui économise le tiers de l'état de régularisation. Et à l'initialisation $F \equiv 0$: tant qu'aucune consolidation n'a eu lieu, le rappel élastique est nul et la carte apprend en descente de gradient pure.

Un dernier écart, purement numérique, mérite d'être signalé car il éclaire les mesures de parité : la carte met chaque poids à jour sur place puis propage

TABLE 1 – Adaptation du modèle sous contrainte embarquée. La pénalité d’(1) est conservée à l’identique ; ce sont son estimateur d’importance et son régime d’optimisation qui sont simplifiés.

	PC (PyTorch)	Carte (ewc_head.c)
Estimateur d’importance	carré du gradient, éq. (2)	magnitude W^2 , éq. (4)
Données requises	un lot étiqueté par tâche	aucune
Accumulation	remplacement à chaque consolidation	moyenne mobile, $\alpha = 0.99$
Portée de la pénalité	poids et biais	poids seulement
Optimisation	lots, momentum, plusieurs époques	un échantillon, sans momentum, une passe
λ	1000	400
Déclenchement	fin de tâche	drapeau de trame, ou détection de dérive
État initial de F	premier calcul en fin de tâche 1	nul, donc pas de rappel avant consolidation

l’erreur vers la couche précédente avec le poids *déjà modifié*, là où le cadre PC utilise partout les poids d’avant le pas. En inférence gelée, les deux plateformes exécutent la même fonction et la parité est exacte ; en régime de mise à jour, cet écart et le lot unitaire suffisent à expliquer qu’elle ne l’est plus tout à fait.

3.2 Modèle EWC et portage C

Le modèle est un perceptron multicouche à tête EWC (`EWCMlpMulticlass`, $k \rightarrow 32 \rightarrow 16 \rightarrow 2$) entraîné en régime incrémental selon la Section 3.1. La tête est portée en C (`ewc_head.c`) pour la carte : le *forward pass* et la mise à jour Stochastic Gradient Descent (SGD) s’exécutent tous deux sur le Cortex-M4 en FP32 (le F439ZI ne dispose pas de NPU). Les poids sont exportés du checkpoint PyTorch vers un en-tête C généré (`model_weights_ewc.h`) — jamais édité à la main — de sorte que la carte et le PC partagent *exactement* les mêmes paramètres.

3.3 Protocole apparié PC↔carte

Pour rendre la comparaison honnête, la carte et le PC consomment la même séquence : même graine (`seed = 42`), même ordre d’échantillons, même normalisation (les indices de features et les tableaux sont produits par une source unique). Deux protocoles sont évalués : *gelé* (inférence seule, poids figés) et *en ligne* (inférence puis mise à jour continue). La *parité* est le taux d’accord prédiction-à-prédiction entre les deux plateformes.

3.4 Formulation de la quantification

Le mapping et son échelle. Quantifier un tenseur, c'est remplacer ses valeurs réelles par des entiers sur une grille régulière, plus une échelle qui rend cette grille au monde flottant. Pour un poids sur b bits en représentation symétrique signée, avec $q_{\max} = 2^{b-1} - 1$:

$$Q_{ji} = \text{clip}\left(\text{round}\left(\frac{W_{ji}}{s_j}\right), -q_{\max}, q_{\max}\right), \quad \widehat{W}_{ji} = s_j Q_{ji}. \quad (5)$$

Tout le comportement du schéma tient dans le choix de s_j . Nous en comparons trois, qui forment une seule famille et non des méthodes séparées :

$$s_j = \underbrace{\frac{1}{128}}_{\text{figée}} \quad \text{ou} \quad \underbrace{\frac{\max_{j,i} |W_{ji}|}{q_{\max}}}_{\text{par-tenseur}} \quad \text{ou} \quad \underbrace{\frac{\max_i |W_{ji}|}{q_{\max}}}_{\text{par-canal de sortie}}. \quad (6)$$

L'échelle figée est celle du noyau embarqué d'origine. Elle ne dépend pas des poids : tout coefficient de module supérieur à $127/128$ est écrasé sur la borne de la grille, et tous les coefficients plus petits que le pas se confondent avec zéro.

La couche entière. Les activations reçoivent le même traitement, avec une échelle s_a issue d'une calibration. Une couche s'écrit alors comme une accumulation entière suivie d'un unique retour au flottant :

$$\text{acc}_j = \sum_i Q_{ji} q_i^a, \quad y_j = \text{acc}_j \cdot s_j \cdot s_a + b_j. \quad (7)$$

C'est dans (7) que se lit la granularité : l'indice j porté par s_j signifie qu'un neurone de sortie dont les poids sont de faible amplitude reçoit sa propre échelle, au lieu de partager celle du coefficient le plus grand de toute la matrice. La calibration retenue est un maximum absolu par couche, mesuré sur un lot en précision flottante ($s_a = \text{act_max}/q_{\max}^a$).

Ce que le schéma d'origine prescrit, et ce que nous en gardons. La formulation entière de référence [4, 6] vise une inférence *entièrement* entière : le retour à l'échelle suivante s'y fait par un multiplicateur entier et un décalage, les biais sont portés en `int32` à l'échelle du produit, et un zéro-point décale la grille des activations. Notre cible n'a pas les mêmes contraintes : le Cortex-M4F possède une unité flottante matérielle, et la tête ne compte que trois couches. Nous conservons donc l'accumulation entière et les échelles par-canal, mais nous revenons au flottant entre les couches. Le tableau 2 récapitule les écarts.

TABLE 2 – Schéma entier de référence et mise en œuvre embarquée. Les écarts sont dictés par la cible (unité flottante présente, réseau court) et non par une simplification de commodité.

	Référence [4]	Noyau (<code>ewc_head_int8_v2.c</code>)	v2
Retour d'échelle	multiplicateur entier et décalage	déquantification flottante sur l'unité matérielle	
Biais	<code>int32</code> à l'échelle du produit	flottants exacts, jamais quantifiés	
Zéro-point	sur les activations et les poids	poids symétriques sans zéro-point ; activations symétriques, affine en option	
Granularité	par-tenseur, par-canal en extension	par-canal de sortie par défaut	
Calibration	histogramme ou centile	maximum absolu par couche sur un lot	
Fusion normalisation	requis avant quantification	sans objet, le réseau n'en contient pas	
Accumulateur	<code>int32</code>	<code>int32</code> , élargi en <code>int64</code> pour la variante Q15	

Axe du format : l'échelle d'ablation. Le noyau embarqué d'origine cumule quatre écarts à (5)–(7) : échelle figée, arrondi remplacé par une troncature vers zéro, accumulateur `int16` qui reboucle silencieusement, et activations repliées sur une grille Q7. Plutôt que de les corriger d'un bloc, nous parcourons une trajectoire dans la famille (6), chaque marche n'activant qu'un changement :

- `legacy_c` : échelle figée, troncature, accumulateur `int16` (le noyau v1 d'origine sur carte) ;
- `fix_acc32` : accumulateur élargi à `int32`, tout le reste inchangé ;
- `per_tensor_calib` : échelle par-tenseur *calibrée* et arrondi au plus proche ;
- `per_channel_int8` : échelle *par-canal* de sortie ;
- `q15` : grille 16 bits, $q_{\max} = 32767$ (RAM $\div 2$ au lieu de $\div 4$).

Le noyau v2 du firmware implémente les deux dernières marches, avec une variante Q15 optionnelle (`-DEWC_INT8_Q15`) ; le chemin v1 reste inchangé pour permettre la comparaison sur la même carte. L'ablation est *émulée bit-à-bit sur PC* (`src/utils/int8_c_emulation.py`) afin d'isoler la contribution de chaque correctif indépendamment de l'accès à la carte. Les échelles ne sont pas recalculées à bord : elles sont produites sur PC par les mêmes fonctions que l'émulateur, puis écrites dans un en-tête C généré, ce qui rend la parité vraie *par construction* plutôt que constatée après coup.

Où s'arrête la quantification en régime d'apprentissage. Un point de portée doit être posé avant de lire les cellules « en ligne » de la Section 5,

car il en commande l'interprétation. Le noyau entier n'expose qu'une passe avant : il n'existe pas de descente de gradient dans le domaine quantifié. La carte conserve donc une tête FP32 *maîtresse*, sur laquelle s'appliquent (3) et (4), et dont la représentation entière n'est qu'une *vue*, reconstruite par (5) après chaque pas. Deux conséquences en découlent, et nous les assumons plutôt que de les taire. D'abord la division par quatre de la mémoire des poids vaut pour l'inférence seule : en régime d'apprentissage, la carte porte simultanément la tête flottante, sa vue entière, l'importance et l'ancrage, soit davantage que le FP32 seul. Ensuite la requantification de toutes les matrices à chaque échantillon entre dans le budget de latence, ce que la Section 5.4 chiffre. Autrement dit, ce travail établit une quantification *compatible* avec l'apprentissage incrémental — l'inférence est entière, l'apprentissage continue de fonctionner à travers elle — et non un apprentissage *conduit* en arithmétique entière, qui reste un problème ouvert.

Ce que coûte le par-canal. Une échelle par neurone de sortie n'est pas gratuite : elle ajoute un tableau de flottants parallèle à chaque matrice, soit une valeur sur quatre octets par ligne de sortie. Sur la tête portée, ces échelles ramènent le gain mémoire réel des poids sous le facteur quatre annoncé par le seul changement de type. C'est le prix de la robustesse : le par-canal isole les neurones de grande amplitude, qui autrement imposeraient leur échelle à toute la couche.

3.5 Axe du moment : quand quantifier

À format fixé, la même tête peut être quantifiée à trois instants. *Avant* l'entraînement, par QAT : un *fake-quant* inséré dans la boucle fait apprendre les poids sous la contrainte de précision réduite. *Après*, par PTQ : le modèle FP32 entraîné est converti une fois pour toutes. Et *les deux*, c'est-à-dire un entraînement QAT dont les poids sont ensuite exportés vers le kernel entier calibré. Ce dernier chemin est le seul fidèle au déploiement réel, puisque c'est bien le noyau embarqué qui exécute l'inférence, et non la simulation de quantification du cadre d'entraînement. Les trois moments sont comparés à modèle, jeu et graine identiques.

3.6 Axe de la profondeur : bits, granularité, symétrie

Sous l'INT8, trois paramètres décrivent un schéma : le nombre de *bits de poids* (4, 2, ternaire, binaire), la *granularité* de l'échelle (par-tenseur ou par-canal de sortie) et la *symétrie* (symétrique, ou affine avec zéro-point). Les deux premiers se lisent directement dans (5) et (6) : descendre en profondeur, c'est diminuer $q_{\max}$, et changer de granularité, c'est changer la portée du maximum qui définit s_j .

Aux très basses profondeurs, la grille linéaire cède la place à deux schémas dédiés, que nous reprenons tels quels. En ternaire [10], un seuil par canal décide quels poids survivent, et l'échelle est la moyenne des seuls poids retenus :

$$\begin{aligned}\Delta_j &= 0.7 \cdot \overline{|W_{j\cdot}|}, & Q_{ji} &= \text{sign}(W_{ji}) \cdot \mathbf{1}[|W_{ji}| > \Delta_j], \\ \alpha_j &= \overline{\{|W_{ji}| : |W_{ji}| > \Delta_j\}}.\end{aligned}\tag{8}$$

En binaire [12], plus aucun poids n'est mis à zéro et l'échelle est la moyenne des modules du canal : $Q_{ji} = \text{sign}(W_{ji})$, $\alpha_j = \overline{|W_{j\cdot}|}$. Ces deux schémas portent une échelle par canal *par construction*, si bien que l'axe de la granularité cesse de s'appliquer une fois qu'on les atteint.

La symétrie, elle, concerne les activations. Après une fonction d'activation positive, la moitié négative d'une grille symétrique ne sert à rien, ce qui coûte d'autant plus cher que les bits sont rares. La variante affine décale la grille d'un zéro-point :

$$q = \text{round}\left(\frac{a}{s_a}\right) + z, \quad \hat{a} = (q - z) s_a.\tag{9}$$

Les poids, dont la distribution est centrée, restent symétriques sans zéro-point dans tous les schémas que nous évaluons.

Un point mérite enfin d'être posé, car il commande l'interprétation des gains annoncés : tant que les poids restent rangés dans un conteneur `int8_t`, réduire le nombre de bits ne libère *aucun* octet. Le gain mémoire n'existe qu'avec un *bit-packing* effectif, qui se paie en revanche par un dépacking à l'exécution. Nous distinguons donc systématiquement la RAM *théorique* du schéma et la RAM *mesurée* du binaire.

3.7 Méthodologie de mesure

Les grandeurs rapportées en Section 7 appellent trois instrumentations distinctes.

Mémoire. La RAM d'un binaire n'est pas l'empreinte de ses poids. Nous rapportons la *RAM totale* = `.data` + `.bss` + pic de pile, les deux premiers termes étant lus dans le binaire et le troisième mesuré à l'exécution par marquage de la pile, en retenant le maximum entre la phase d'inférence et la phase de mise à jour.

Latence. Le compteur de cycles Data Watchpoint and Trace (DWT) chronomètre le noyau au cycle près. Pour le chemin INT8, il est instrumenté en *trois segments* — déquantification, Multiply-Accumulate (MAC) entier, requantification — afin de répondre non pas « l'INT8 est-il plus lent ? » mais « quel poste coûte ? ». Les cycles sont rapportés bruts, la conversion en microsecondes tronquant l'information à cette échelle.

Énergie. Le courant est mesuré à la sonde X-NUCLEO-LPM01A par trois méthodes indépendantes : régression du courant en fonction de la cadence d’inférence, différence contre un repos Wait For Interrupt (WFI), et lot d’inférences par trame isolant le calcul du coût de la liaison. Nous les rapportons *côte à côte* sans jamais les moyenner : elles ne mesurent pas la même frontière, et leur désaccord est lui-même un résultat. Toute cellule non mesurée est reportée « à mesurer » plutôt qu’estimée.

4 Dispositif expérimental

Cible matérielle. NUCLEO-F439ZI (Cortex-M4 @ 180 MHz, 256 kB SRAM = 192 kB + 64 kB Core Coupled Memory (CCM), pas de NPU). Les latences sont mesurées par le compteur de cycles DWT ; l’empreinte statique est lue dans le binaire et complétée par le pic de pile (§3). Le budget de latence est 100 ms par cycle inférence + mise à jour (Gap 2). Les échantillons transitent par la liaison Universal Asynchronous Receiver-Transmitter (UART), trame par trame, chaque trame étant protégée par un CRC : un taux d’erreur nul est la condition pour que les prédictions comparées soient bien celles de la carte.

Jeux de données et scénarios CL. Deux jeux industriels aux scénarios contrastés : *Pronostia* (roulements FEMTO [11], *class-incremental* par condition normale/défaut) et *Monitoring* (équipements industriels, *domain-incremental* par type de machine). Deux conditions de features sont considérées : **5feat** (sous-ensemble embarqué) et **all** (features natives). Sur Monitoring, **5feat** et **all** coïncident (4 features natives).

Banc d’énergie. Le courant est relevé par une sonde X-NUCLEO-LPM01A, la carte étant alimentée par la sonde elle-même. Les campagnes sont conduites à cadence d’inférence imposée et fenêtre fixe, avec répétitions, de sorte que l’écart entre deux cellules soit imputable au modèle et non à l’hôte. Un balayage de fréquence d’horloge complète le dispositif, la fréquence du bus périphérique étant maintenue constante pour ne pas déplacer le débit de la liaison série.

Origine des mesures. Le Tableau 3 récapitule les campagnes et, pour chacune, sa nature. Cette distinction gouverne la lecture de tout l’article : une valeur *mesurée sur carte* et une valeur *émulée sur PC* ne s’opposent pas en fiabilité — l’émulateur reproduit exactement l’arithmétique du noyau C — mais elles ne répondent pas à la même question.

Conditions identiques et règle d’honnêteté. Toutes les comparaisons partagent graine, ordre et normalisation (§3). La campagne carte v2 couvre

TABLE 3 – Origine et nature des mesures rapportées.

Campagne	Objet	Nature
<code>exp_S36</code>	Parité FP32, INT8 legacy, latence, RAM	mesuré carte
<code>exp_S39_ablation</code>	Échelle d’ablation, diagnostic causal	émulé PC
<code>exp_S40_board_v2</code>	Récupération, kernel v2 calibré	mesuré carte
<code>exp_S46_board</code>	Moment de quantification, A/B	mesuré carte
<code>exp_S47_depth</code>	Profondeur et granularité	émulé PC
<code>exp_S48_summary</code>	Bit-packing, <code>.bss</code> réelle	mesuré carte
<code>exp_S49_ram</code>	RAM totale par composante	mesuré carte
<code>exp_S50_int8_latency</code>	Latence par segment de noyau	mesuré carte
<code>exp_S53_*</code>	Énergie, cadence, fréquence	mesuré carte

TABLE 4 – Parité FP32 PC↔carte et performance (*mesuré sur carte*, `exp_S36`). Le F1 est celui de la carte ; il égale celui du PC aux décimales rapportées, la parité gelée valant 1.000. Monitoring n’a que quatre features natives, ses conditions `5feat` et `all` coïncident donc et ne donnent qu’une ligne.

Jeu	Cond.	F1 carte	Parité gelée	Parité en ligne
Pronostia	<code>5feat</code>	0.9164	1.000	0.9754
Pronostia	<code>all</code>	0.9180	1.000	0.9626
Monitoring	<code>5feat</code>	0.9194	1.000	0.9887

les quatre cellules per-canal (deux jeux $\times$ deux protocoles) ; le format `q15` sur carte n’a pas été streamé. Toute grandeur non mesurée est reportée « à mesurer », sans valeur inventée ni estimation substituée ; le registre complet de ces cellules, avec la raison de chacune, est donné en Annexe C. Les sigles employés dans l’article sont développés à leur première occurrence et rassemblés dans la liste des acronymes, page 32.

5 Résultats

5.1 Parité FP32 PC↔carte (mesuré sur carte)

En FP32, la parité prédiction-à-prédiction est *exacte* en mode gelé (1.000) sur les deux jeux, et quasi exacte en mode en ligne (0.9754 Pronostia, 0.9887 Monitoring) — les écarts en ligne proviennent du `float32` de la carte contre le `float64` du PC sur les frontières de décision. Le Tableau 4 récapitule ; la Figure 1 l’illustre.

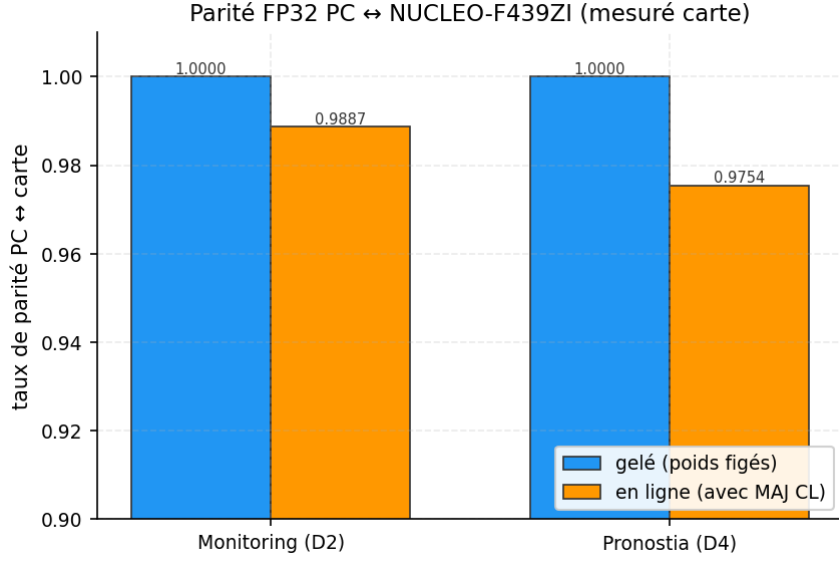


FIGURE 1 – Parité FP32 PC↔carte : gelé (1.000) vs en ligne. *Mesuré sur carte* (`exp_S36_parity_*.json`).

TABLE 5 – INT8 *legacy* vs FP32 sur carte (*mesuré*, `exp_S36`). RAM $\div 4$ mais F1 effondré.

Jeu	F1 FP32	F1 INT8 legacy	Accord INT8/FP32	RAM $\div$
Pronostia	0.9164	0.138	0.7364	$\times 4$
Monitoring	0.9194	0.1337	0.5955	$\times 4$

Latence et RAM (Gap 2/Gap 3). Toutes les latences carte restent trois ordres de grandeur sous le budget : inférence $50\ \mu\text{s}$ (Pronostia) / $48\ \mu\text{s}$ (Monitoring) ; inférence + mise à jour continue $251\ \mu\text{s}$ / $239\ \mu\text{s}$, contre $\approx 33\text{--}35\ \mu\text{s}$ pour l’inférence PC de référence. L’empreinte `.bss` varie de 100–145 kB, sous les 256 kB disponibles. La Figure 2 situe ces valeurs face à la ligne des 100 ms.

5.2 Effondrement de la PTQ naïve (mesuré sur carte)

La quantification INT8 *legacy* de la tête EWC binaire s’effondre sur la carte : le F1 chute de ≈ 0.92 (FP32) à 0.138 (Pronostia) et 0.1337 (Monitoring), pour un accord INT8↔FP32 de seulement $\approx 0.60\text{--}0.74$. La RAM des poids est bien divisée par 4, mais la métrique est détruite (Tableau 5).

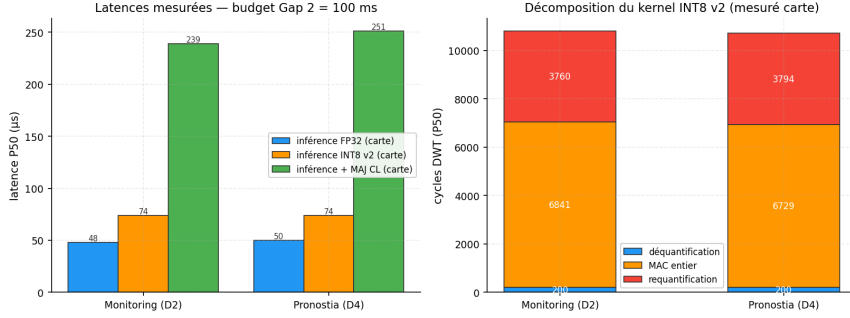


FIGURE 2 – Latences carte (inférence vs inférence + mise à jour) contre le budget Gap 2 (100 ms). *Mesuré sur carte (exp_S36_summary.json).*

TABLE 6 – Échelle d’ablation INT8 (F1, *émulé PC bit-exact*, exp_S39_ablation).

Schéma	Pronostia F1	Monitoring F1
FP32 (réf.)	0.9616	0.9194
legacy_c	0.0663	0.1178
per_tensor_calib	0.9462	0.9201
per_channel_int8	0.9426	0.9187
q15	0.9616	0.9194

5.3 Diagnostic de l’effondrement (émulé PC bit-exact)

L’effondrement mesuré ne dit pas *ce qui* l’a causé. Une ablation *émulée bit-à-bit sur PC* — référence exacte de l’arithmétique du kernel — isole la contribution de chaque correctif (Tableau 6, Figure 3). Le facteur dominant est l’échelle *par-tenseur calibrée* sur accumulateur `int32` : `per_tensor_calib` restaure +0.88 de F1 (Pronostia 0.9462, Monitoring 0.9201). Le format `q15` égale exactement le FP32.

La cause est donc identifiée, et l’émulateur *prédit* la récupération. Reste à savoir si le noyau corrigé la produit effectivement une fois flashé : c’est l’objet de la sous-section suivante, où la question passe de l’émulation à la mesure.

5.4 Récupération mesurée sur carte (kernel v2 calibré)

La récupération n’est plus une prédiction d’émulateur : le kernel v2 à échelle calibrée a été flashé sur la NUCLEO-F439ZI et streamé sur les quatre cellules per-canal (deux jeux $\times$ deux protocoles). Le F1 revient au niveau FP32 — 0.9173 contre 0.9194 en FP32 sur Monitoring, 0.8995 contre 0.9164 sur Pronostia — alors que la RAM des poids reste divisée par 4. La parité vis-à-vis de l’émulateur bit-exact est *exacte* en mode gelé (1.000), ce qui valide

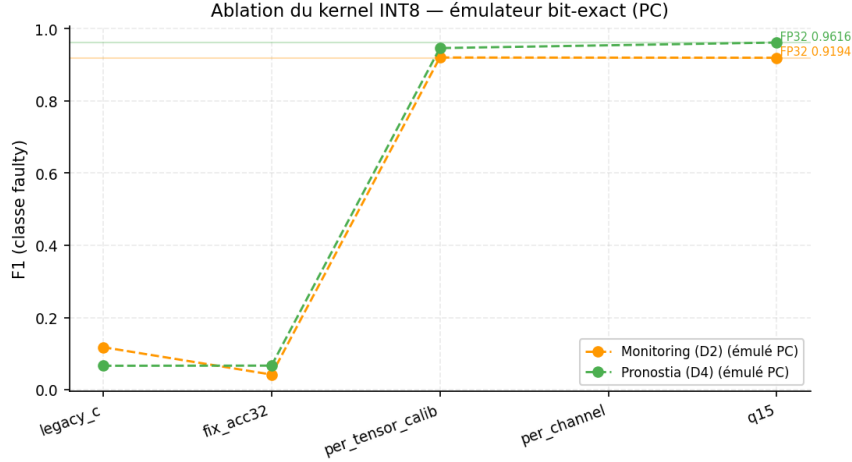


FIGURE 3 – Échelle d’ablation `legacy_c`→`per_tensor_calib`→`per_channel`→`q15`. Émulé PC bit-exact (`exp_S39_ablation`).

TABLE 7 – Récupération INT8 *mesurée sur carte* (kernel v2 per-canal, `exp_S40_board_v2`). Parité = accord avec l’émulateur bit-exact ; accord = INT8 contre FP32 sur les mêmes échantillons.

Jeu	Protocole	F1	Parité	Accord	Latence (μ s)	.bss (B)
Monitoring	gelé	0.9173	1.000	0.9996	65	101236
Monitoring	en ligne	0.9016	0.9885	0.8337	577	101236
Pronostia	gelé	0.8995	1.000	0.9951	68	106152
Pronostia	en ligne	0.9212	0.9736	0.8100	606	106152

le portage sans perte, et aucune erreur CRC n’a été observée (Tableau 7).

Le surcoût de latence est bien plus lourd en apprentissage qu’en inférence. La colonne des latences du Tableau 7 porte le chiffre le plus net de cette campagne, et il ne concerne pas le mode gelé. En inférence seule, l’INT8 coûte 65 et 68 μ s contre 48 et 50 en FP32, soit l’écart d’environ moitié que la Section 7 décompose poste par poste. En apprentissage continu, l’écart change d’ordre : 577 et 606 μ s contre 239 et 251 en FP32, soit un facteur 2.4. La cause est celle exposée en §3 : chaque pas de gradient s’applique à la tête FP32 maîtresse, et la vue entière doit être intégralement reconstruite derrière lui. Le budget Gap 2 reste tenu avec deux ordres de grandeur de marge, mais la lecture est sans ambiguïté — **c’est précisément dans le régime que l’apprentissage continu impose que la quantification coûte le plus cher**. Une requantification restreinte aux lignes effectivement modifiées par le pas est la voie naturelle pour ramener ce facteur près de un ; elle reste à

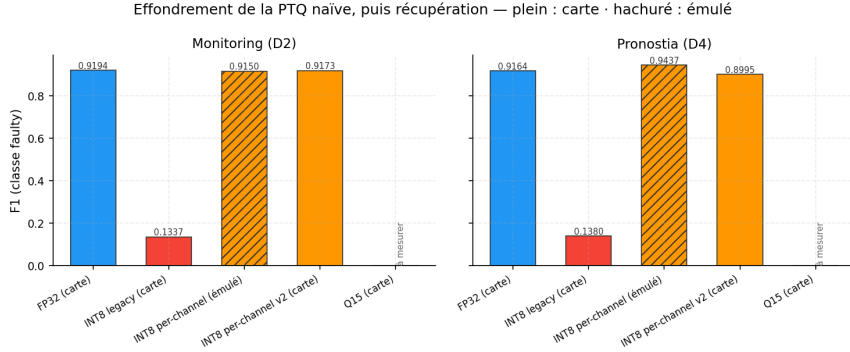


FIGURE 4 – INT8 vs FP32 : effondrement *legacy* puis récupération. Effondrement et récupération sont tous deux *mesurés sur carte* (Sprints 36 et 40) ; la colonne per-canal *émulée PC* (Sprint 39) est rappelée en hachuré. Le format **q15** sur carte reste « à mesurer ».

implémenter.

Deux nuances méritent d’être dites. D’une part l’accord $\text{INT8} \leftrightarrow \text{FP32}$ chute en mode en ligne (0.8337 et 0.8100 contre > 0.99 en gelé) : la mise à jour continue fait diverger les deux trajectoires de poids, sans que la métrique finale en souffre. D’autre part le format **q15** et la reprise A/B du schéma `int8_legacy` sous le kernel v2 n’ont pas été streamés : ces cellules restent « à mesurer », conformément à la règle « aucun chiffre inventé ».

5.5 Compromis RAM $\times$ F1 $\times$ latence

La Figure 5 synthétise le compromis : le kernel calibré ramène la métrique au niveau FP32 tout en conservant le gain RAM ($\text{INT8} \div 4$, $\text{Q15} \div 2$), sans coût de latence prohibitif. L’étoile marque le point réellement flashé (kernel v2 per-canal) ; les autres points sont émuls. Le gain de latence espéré, lui, n’a pas lieu — nous y revenons en Section 8.

6 Deux autres axes de quantification : le moment et la profondeur

Les sections précédentes font varier la *calibration* du kernel à format fixe. Deux autres degrés de liberté commandent le résultat : *quand* on quantifie et *combien de bits* on garde. Les trois placements comparés ici et les paramètres d’un schéma sub-INT8 sont définis en §3.

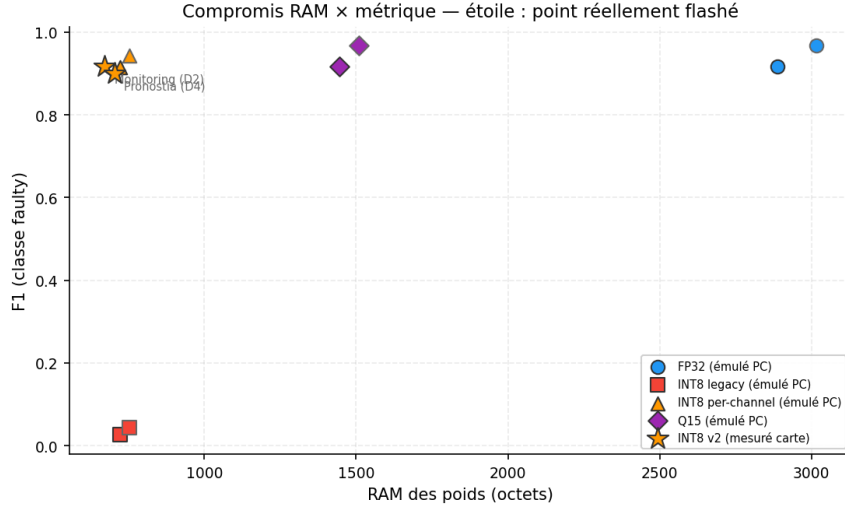


FIGURE 5 – Front de compromis RAM $\times$ F1 $\times$ latence. Sources mixtes *mesuré/émulé* annotées par point.

6.1 Moment : avant, après, ou les deux

Trois placements sont possibles pour la même tête EWC. *Avant* l’entraînement (QAT, fake-quant dans la boucle) ; *après* (PTQ sur un modèle FP32 déjà entraîné) ; et *les deux*, c’est-à-dire un entraînement QAT dont les poids sont ensuite exportés vers le kernel entier calibré — seul ce dernier chemin est fidèle au déploiement réel.

En émulation, les trois moments préservent l’Area Under the ROC Curve (AUROC) sur Monitoring (0.974477 avant, 0.974649 après, 0.974663 les deux, contre 0.97464 en FP32) : à cette échelle de modèle, aucun moment n’est disqualifiant. La question tranchée l’est *sur carte*. Le chemin « les deux » a été flashé et streamé : il donne 0.9213 de F1 sur Monitoring et 0.9072 sur Pronostia, contre respectivement 0.9173 et 0.8995 pour la PTQ calibrée seule (Section 5.4).

L’écart, 0.004 et 0.0077, est réel mais faible. La lecture honnête est donc que **la calibration du noyau récupère l’essentiel, et que le QAT n’ajoute pas de gain décisif** sur cette tête : quand le pipeline d’entraînement ne peut pas être modifié, une PTQ correctement calibrée suffit. Ce constat contredit la lecture répandue selon laquelle un effondrement en PTQ appellerait nécessairement un réentraînement.

6.2 Profondeur : jusqu’où descendre sous 8 bits

Sous les 8 bits, la question devient : à partir de quelle profondeur la métrique décroche, et la granularité du facteur d’échelle change-t-elle cette limite ? Le balayage émulé répond aux deux.

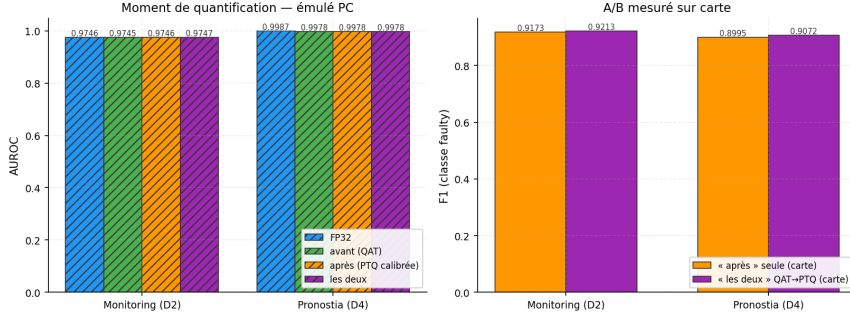


FIGURE 6 – Moment de quantification. À gauche, les trois moments *émulés PC* (hachuré) ; à droite, l’A/B *mesuré sur carte* entre la PTQ calibrée seule et le chemin QAT→PTQ.

La limite dépend du jeu ; la grille complète est en Annexe A (Tableau 9). Sur Monitoring, la dégradation reste contenue jusqu’au binaire ($\Delta\text{AUROC} = -0.0117$). Sur Pronostia, le binaire décroche (-0.0275) mais le ternaire tient (-0.0153) : lue en AUROC, la frontière exploitable serait donc le *ternaire*, le binaire n’étant défendable que sur le jeu le plus facile.

La frontière dépend de la métrique de sélection. Cette lecture ne survit pas à la confrontation avec le F1 *mesuré sur carte* des mêmes schémas, que le Tableau 8 place à côté de l’AUROC. Sur Pronostia, le ternaire conserve une AUROC de 0.99907 mais son F1 tombe à 0.379 ; le binaire garde une AUROC de 0.96616 pour un F1 *nul*, c’est-à-dire un modèle qui ne prédit plus jamais la classe de défaut. L’ordre des scores reste bon là où le seuil de décision, lui, ne l’est plus : une métrique de rang est insensible au déséquilibre des classes, un F1 ne l’est pas. Sur les deux jeux, le seul schéma sub-INT8 qui préserve réellement la métrique d’exploitation est l’**INT4** (0.921 et 0.9887).

Nous rapportons les deux lectures plutôt que d’en choisir une, car leur écart est lui-même le résultat : **ce n’est pas la profondeur seule qui fixe la frontière, c’est le couple profondeur–métrique de sélection**. Un travail qui ne rapporterait que l’AUROC conclurait au ternaire et déploierait, sur Pronostia, un détecteur qui ne détecte rien.

La granularité, elle, repousse le décrochage. À 2 bits sur Pronostia, passer d’une échelle par-tenseur à une échelle par-canal fait passer ΔAUROC de -0.0464 à -0.0086 . Un point négatif mérite d’être rapporté aussi : le *zéro-point* affine ne rachète rien — à 2 bits sur Monitoring il dégrade même nettement (-0.0619 contre -0.0069 en symétrique), et le Tableau 10 montre que le constat vaut aux trois profondeurs testées.

Le gain RAM n’existe que packé. Le point crucial est que la RAM annoncée par ces schémas est *théorique* tant que les poids restent stockés

TABLE 8 – Schémas sub-INT8 *mesurés sur carte* : AUROC et F1 de la classe **faulty** sur les mêmes cellules. L’AUROC reste haute là où le F1 décroche, la profondeur ne déplaçant pas les deux métriques ensemble.

Profondeur	Monitoring		Pronostia	
	AUROC	F1	AUROC	F1
int4	0.98787	0.9210	0.99995	0.9887
ternaire	0.98409	0.9042	0.99907	0.3790
binaire	0.96784	0.6878	0.96616	0.0000

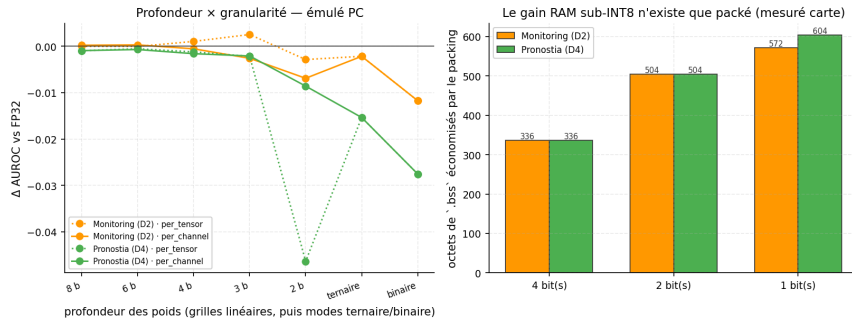


FIGURE 7 – Profondeur sub-INT8. À gauche, ΔAUROC par profondeur et granularité (*émulé PC*) — la lecture en F1 mesuré sur carte, qui déplace la frontière, est au Tableau 8 ; à droite, les octets de **.bss** réellement économisés par le bit-packing (*mesuré sur carte*).

dans un conteneur **int8**. Mesuré sur carte, le **.bss** d’un modèle 4, 2 ou 1 bit *non packé* est strictement identique à celui de l’INT8 : le gain n’apparaît qu’avec un véritable bit-packing, où il croît quand les bits baissent (336, 504 puis 572 octets économisés sur Monitoring). Le dépacking se paie en latence, d’environ 55 μs (de 67 μs non packé à 123 μs packé sur Monitoring) — sans menacer le budget Gap 2. La parité contre l’émulateur reste 1.000 sur les douze cellules mesurées : le schéma est porté sans perte.

7 Budget système : mémoire, latence, énergie, calcul

Un gain de quantification ne se juge pas sur les poids seuls. C’est ici que se tranche l’argument tripartite rappelé en introduction — moins de mémoire, moins de temps, moins d’énergie : cette section rassemble les quatre budgets d’une même tête EWC déployée, tous mesurés sur la même carte, et prend chaque promesse séparément. Les trois instrumentations correspondantes (composantes de la RAM, segmentation du noyau au compteur de

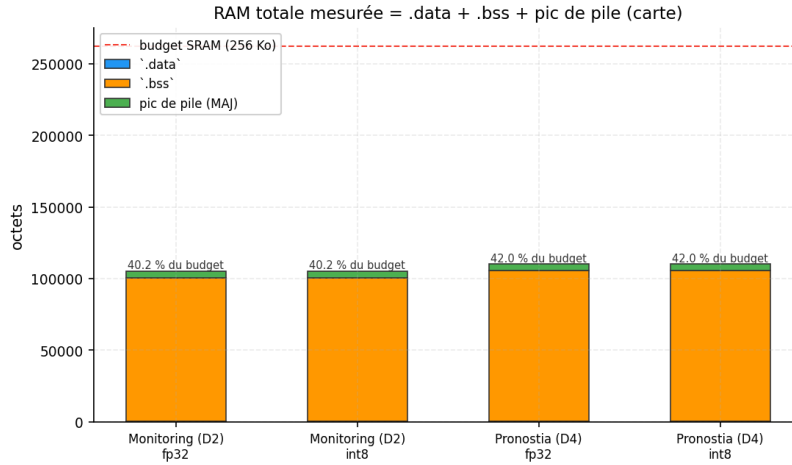


FIGURE 8 – RAM totale *mesurée sur carte* : `.data` + `.bss` + pic de pile, en regard du budget SRAM. Le total INT8 égale le total FP32, alors même que les poids sont divisés par 4.

cycles, estimateurs d'énergie) sont décrites en §3.

7.1 La RAM des poids n'est pas la RAM du système

L'INT8 divise bien par 4 la RAM *des poids* (2688 à 672 octets sur Monitoring, 2816 à 704 sur Pronostia). Ce décompte porte sur les matrices de poids seules, celles que la quantification touche ; les biais restent flottants et exacts (§3), si bien que le nombre de paramètres de la tête, 722 et 754, est légèrement supérieur. Mais la RAM réellement consommée par le système est `.data` + `.bss` + pic de pile, et cette somme ne bouge pratiquement pas : elle passe de 105300 à 105324 octets sur Monitoring, soit un rapport INT8/FP32 de 1.0002. Le détail par composante est en Annexe B (Tableau 11). Les deux configurations occupent 40.17 % et 40.18 % du budget de 256 kB (42.03 % et 42.04 % sur Pronostia).

L'explication est simple et vaut d'être dite clairement : à cette échelle de modèle, les poids ne sont pas le poste dominant — les tampons de flux, la pile et le reste du firmware le sont. Annoncer un « $\div 4$ de RAM » sans préciser *de quelle* RAM on parle serait donc trompeur. Le $\div 4$ est réel et utile, mais il porte sur un poste minoritaire de ce système-ci ; il deviendrait décisif sur un modèle où les poids dominent.

7.2 Où passe la latence INT8

L'instrumentation cycle-à-cycle du kernel INT8 chiffre le paradoxe annoncé. Sur Monitoring, le MAC entier coûte 6841 cycles, la requantification 3760 et la déquantification 200 (respectivement 6729, 3794.5 et 200 sur Pro-

nostia). Le total atteint 74 μ s contre 48 μ s en FP32 sur Monitoring et 50 μ s sur Pronostia.

Le poste décisif n'est donc pas la multiplication : c'est la *requantification*, un tiers du budget, et elle n'existe tout simplement pas dans le chemin FP32. Le MAC entier, lui, n'apporte aucun gain face au FPU matériel du Cortex-M4. Le surcoût INT8 n'est pas une anomalie de mise en œuvre mais la conséquence arithmétique d'un format entier exécuté sur une cible qui calcule nativement en flottant.

7.3 Énergie : trois estimateurs, jamais fusionnés

L'énergie a été mesurée à la sonde X-NUCLEO-LPM01A par trois méthodes indépendantes, que nous reportons *côte à côte* sans jamais les moyenner — leur désaccord est lui-même un résultat. La régression du courant en fonction de la cadence donne 67.65 μ J par inférence en FP32 et 67.47 μ J en INT8 ; le delta contre un repos WFI donne 146.77 μ J et 145.03 μ J ; le lot d'inférences par trame, qui isole le calcul du coût de la trame UART, donne 5.08 μ J. Le Tableau 12 les range côte à côte. L'ordre de grandeur commun aux deux premières et l'écart d'un facteur 10 avec la troisième délimitent exactement ce que chacune mesure : les deux premières intègrent le coût de communication, la dernière non.

Sur la question qui nous occupe, les trois convergent : **l'INT8 ne réduit pas la consommation**. L'écart mesuré est de 0.18 μ J pour une incertitude combinée de 2.34 μ J, soit très en deçà du seuil de signification. Le paradoxe de la latence se double donc d'un paradoxe d'énergie, et la justification de l'INT8 se réduit à la mémoire.

Le balayage de fréquence ouvre en revanche un levier réel. L'énergie par inférence croît monotonement avec l'horloge — 170.05 μ J à 45 MHz, 192.64 μ J à 90 MHz, 214.61 μ J à 180 MHz — pendant que la latence reste très en deçà du budget Gap 2, jusqu'à 198 μ s au point le plus lent. Autrement dit, *la marge de latence est convertible en autonomie* : au régime le plus sobre mesuré, une pile de 2000 mAh tient 72.95 h à une inférence par seconde. Deux cellules restent « à mesurer » : l'énergie par mise à jour continue, dont les régressions ne sont pas publiables en l'état, et le profil par phase.

7.4 Coût de calcul

Analytiquement, la tête compte 722 paramètres sur Monitoring (754 sur Pronostia), pour 672 MAC et 1344 Floating-Point Operation (FLOP) par inférence (704 et 1408). Le compte en *opérations binaires* donne le ratio théorique attendu : 1376256 BOPs en FP32 contre 86016 en INT8, soit un facteur 16 — c'est-à-dire $(32/8)^2$.

Ce facteur 16 est précisément ce que la carte *ne* restitue pas. Confronter les deux colonnes de la Figure 10 est la manière la plus directe de voir

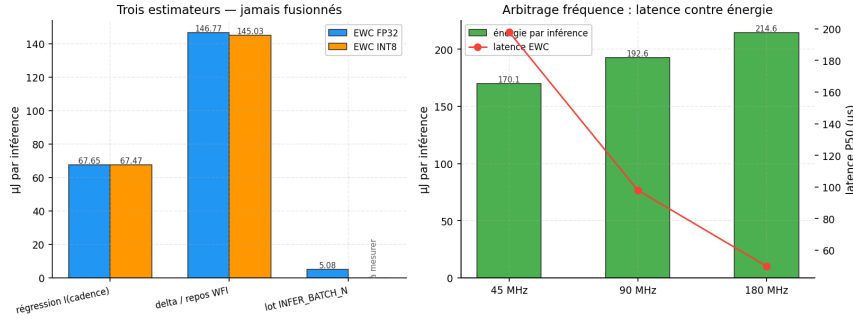


FIGURE 9 – Énergie *mesurée sur carte*. À gauche, les trois estimateurs côte à côte ; à droite, l’arbitrage fréquence, où la marge de latence s’échange contre de l’énergie.

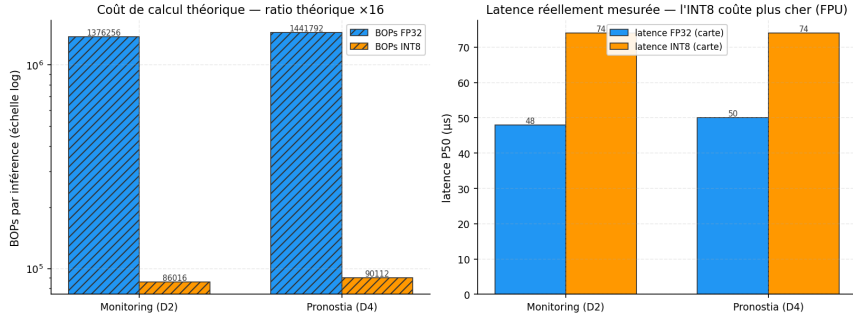


FIGURE 10 – Coût de calcul *théorique* (gauche, hachuré) contre latence *mesurée sur carte* (droite). Le gain de 16 en BOPs ne se traduit par aucun gain de temps.

pourquoi une métrique de coût analytique ne peut pas remplacer une mesure : le Bit Operation (BOP) compte des bits d’opérande, pas des cycles d’une unité flottante existante.

8 Discussion

Paradoxe de la latence INT8, chiffré. La quantification INT8 divise par 4 l’empreinte mémoire des poids mais *n’accélère pas* l’inférence sur le Cortex-M4 — elle la ralentit. L’instrumentation cycle-à-cycle du kernel (Section 7) permet de dire où : sur les deux jeux, le MAC entier coûte en moyenne 6785 cycles et la requantification 3777.25, la déquantification n’en pesant que 200. Le total INT8 s’établit à 74 μs contre 48–50 μs en FP32. Deux causes se cumulent donc : le MAC entier n’a rien à gagner face à un FPU matériel déjà efficace en simple précision, et la requantification — un poste qui n’existe pas dans le chemin flottant — consomme à elle seule un tiers du budget. La piste

directe reste le portage sur des noyaux entiers Single Instruction, Multiple Data (SIMD), tels ceux de Common Microcontroller Software Interface Standard — Neural Network (CMSIS-NN) [8], qui attaquerait le premier poste ; le second appelle plutôt une chaîne à échelles en puissance de deux. Ce bilan porte sur l’inférence. En apprentissage continu, le surcoût est plus lourd encore, d’un facteur 2.4 (Section 5.4), car la reconstruction de la vue entière suit chaque pas de gradient. C’est le régime que revendique l’apprentissage embarqué, et c’est donc là que le paradoxe pèse le plus.

Un paradoxe d’énergie s’ajoute au paradoxe de latence. On pourrait espérer que l’INT8, à défaut d’être plus rapide, soit plus sobre. La mesure dit le contraire : 67.47 μJ par inférence en INT8 contre 67.65 μJ en FP32, soit un écart de 0.18 μJ pour une incertitude combinée de 2.34 μJ . Il n’y a donc pas d’effet mesurable. La conséquence pratique est nette : sur cette cible, **l’INT8 se justifie par la mémoire, ni par la vitesse ni par l’énergie**. Le levier énergétique se trouve ailleurs — dans la fréquence, où la marge de latence dont on dispose largement s’échange contre de l’autonomie.

Honnêteté « mesuré » vs « émulé ». Nous séparons rigoureusement deux natures de résultats. Les parités FP32, l’effondrement INT8 legacy, la récupération par kernel v2, les latences, la RAM et l’énergie sont *mesurés sur carte réelle* (Sprints 36, 40, 46 à 50 et 53). Le diagnostic causal, l’échelle d’ablation et le balayage de profondeur sont *émulés bit-à-bit sur PC* (Sprints 39 et 47) : l’émulateur reproduit exactement l’arithmétique du kernel C, ce qui en fait une référence fiable, mais ce n’est pas une mesure carte. Depuis la campagne v2, la récupération n’appartient plus à la seconde catégorie : elle est établie sur les quatre cellules per-canal, avec une parité gelée de 1.000 contre l’émulateur. Ce qui reste non mesuré — le format **q15** sur carte, l’énergie par mise à jour continue, le profil par phase — est reporté « à mesurer » plutôt qu’estimé.

Limite : l’oubli n’est pas éprouvé sur ces deux jeux. Nos scénarios continus y sont courts (trois tâches) et l’oubli moyen y reste inférieur à 0.01 : ces jeux ne mettent pas la régularisation EWC à l’épreuve. L’oubli catastrophique a été mesuré ailleurs dans le projet, sur CWRU, où il atteint 0.8475 pour EWC contre 0.8578 en apprentissage naïf. Comme ce jeu ne recouvre pas les deux nôtres, nous nous gardons de mélanger les deux constats : la contribution de cet article porte sur le *portage* et la *quantification*, pas sur l’efficacité de la régularisation contre l’oubli.

Portée. Le résultat central — une PTQ naïve peut détruire une tête de décision binaire, et une calibration par-tenseur/par-canal (ou un format Q15)

la restaure — dépasse le cas EWC : il concerne toute tête légère quantifiée en aval d’un extracteur, régime fréquent en TinyML.

9 Conclusion et travaux futurs

Nous avons porté un classifieur EWC de maintenance prédictive sur un microcontrôleur Cortex-M4 et établi, dans un protocole apparié, une parité FP32 PC $\leftrightarrow$ carte exacte en mode gelé, avec une latence inférence + apprentissage continu $\ll$ 100 ms et une empreinte sous 256 kB (Gaps 2 et 3). Sur cette même carte, nous avons *mesuré* l’effondrement de la quantification INT8 naïve de la tête binaire, *isolé sa cause* par ablation émulée bit-exacte — une échelle par-tenseur non calibrée sur accumulateur trop étroit — puis *mesuré sa récupération* par un kernel calibré, avec un F1 revenu à 0.9173 et 0.8995 et une parité gelée de 1.000 contre l’émulateur. Le diagnostic reste émulé, la récupération ne l’est plus.

Deux axes complémentaires précisent ce résultat. Sur le *moment* de la quantification, la calibration du noyau récupère l’essentiel et un entraînement QAT n’ajoute pas de gain décisif : quand le pipeline d’entraînement ne peut pas être modifié, une PTQ correctement calibrée suffit. Sur la *profondeur*, la frontière n’est pas fixée par les bits seuls mais par le couple profondeur-métrique : le ternaire tient en AUROC et décroche en F1 mesuré sur carte, où la frontière redevient l’INT4. La granularité par-canal repousse le décrochage, et le gain mémoire annoncé sous 8 bits n’existe qu’avec un véritable bit-packing — qui se paie environ 55 μ s de dépacking, sans menacer le budget.

Ce que l’INT8 apporte, et ce qu’il n’apporte pas. Le budget système mesuré dissocie les trois promesses habituelles. La RAM des poids est bien divisée par 4, mais la RAM totale du système est inchangée (rapport 1.0002), parce qu’à cette échelle de modèle les poids ne sont pas le poste dominant ; et ce $\div 4$ vaut pour l’inférence, la tête flottante restant en mémoire dès que la carte apprend. La latence *augmente* d’environ la moitié en inférence et d’un facteur 2.4 en apprentissage continu, la requantification — un poste absent du chemin flottant — consommant à elle seule un tiers du budget d’inférence. L’énergie, enfin, ne bouge pas : l’écart mesuré est un ordre de grandeur sous son incertitude. Sur cette cible, l’INT8 se justifie donc par la mémoire seule, et le levier énergétique se trouve ailleurs — dans la fréquence, où la marge de latence dont nous disposons largement s’échange contre de l’autonomie.

Travaux futurs. Le paradoxe de latence oriente vers des noyaux SIMD entiers (CMSIS-NN) pour le poste MAC, et vers une chaîne à échelles en puissance de deux pour le poste de requantification ; c’est la voie la plus directe pour que le gain mémoire cesse de se payer en temps. Le régime d’apprentissage appelle en propre deux pistes : une requantification restreinte

aux lignes modifiées par le pas de gradient, qui attaque directement le facteur 2.4, et une descente de gradient conduite dans le domaine entier, seule manière de faire porter le gain mémoire sur l'apprentissage lui-même et non sur la seule inférence. Le constat « la RAM totale ne bouge pas » appelle par ailleurs une réplication sur un modèle où les poids *dominant* l'empreinte, régime dans lequel le $\div 4$ redeviendrait décisif. Enfin, trois cellules restent explicitement « à mesurer » : le format **q15** sur carte, l'énergie par mise à jour continue et le profil énergétique par phase. Nous les laissons vides plutôt que de les estimer — la même règle qui, tout au long de cet article, sépare ce qui est mesuré sur carte de ce qui est émulé sur PC.

Disponibilité du code et des données

Le firmware C, les noyaux de quantification, l'émulateur bit-exact, les pilotes de campagne et les scripts de figures appartiennent au dépôt du projet CL-Embedded. Chaque valeur rapportée ici provient d'un fichier de résultats versionné, rassemblé sans recalcul dans un agrégat unique dont les figures et les tables sont dérivées : aucune valeur n'est saisie à la main dans le texte ni dans les figures. Les jeux de données sont publics et cités en §4.

Remerciements

Ce travail a été mené à l'ISAE-SUPAERO (DISC), en collaboration avec l'ENAC (LII) et Edge Spectrum. L'auteur remercie ses encadrants pour leurs relectures et leurs orientations sur la quantification embarquée et sur la conduite des campagnes de mesure.

Références

- [1] Simone Benatti, Fabio Montagna, Victor Kartsch, Abbas Rahimi, Davide Rossi, and Luca Benini. Online learning and classification of EMG-based gestures on a parallel ultra-low power platform using hyperdimensional computing. *IEEE Transactions on Biomedical Circuits and Systems*, 2019.
- [2] Luigi Capogrosso, Federico Cunico, Dong Seon Cheng, Federico Fummi, and Marco Cristani. A machine learning-oriented survey on tiny machine learning. *IEEE Access*, 2023.
- [3] Jhair Hurtado, Alain Salvati, Andrea Cossu, Antonio Carta, and Davide Bacciu. Continual learning for predictive maintenance : Overview and challenges. *Intelligent Systems with Applications*, 2023.
- [4] Benoit Jacob, Skirmantas Kligys, Bo Chen, Menglong Zhu, Matthew Tang, Andrew Howard, Hartwig Adam, and Dmitry Kalenichenko.

- Quantization and training of neural networks for efficient integer-arithmetic-only inference. In *IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2018.
- [5] James Kirkpatrick, Razvan Pascanu, Neil Rabinowitz, Joel Veness, Guillaume Desjardins, Andrei A. Rusu, Kieran Milan, John Quan, Tiago Ramalho, Agnieszka Grabska-Barwinska, Demis Hassabis, Claudia Clopath, Dharshan Kumaran, and Raia Hadsell. Overcoming catastrophic forgetting in neural networks. *Proceedings of the National Academy of Sciences*, 2017.
 - [6] Raghuraman Krishnamoorthi. Quantizing deep convolutional networks for efficient inference : A whitepaper. arXiv :1806.08342, 2018.
 - [7] Young D. Kwon, Jagmohan Chauhan, Hong Jia, Stylianos I. Venieris, and Cecilia Mascolo. LifeLearner : Hardware-aware meta continual learning system for embedded computing platforms. In *ACM Conference on Embedded Networked Sensor Systems (SenSys)*, 2023.
 - [8] Liangzhen Lai, Naveen Suda, and Vikas Chandra. CMSIS-NN : Efficient neural network kernels for Arm Cortex-M CPUs. arXiv :1801.06601, 2018.
 - [9] Matthias De Lange, Rahaf Aljundi, Marc Masana, Sarah Parisot, Xu Jia, Aleš Leonardis, Gregory Slabaugh, and Tinne Tuytelaars. A continual learning survey : Defying forgetting in classification tasks. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 2021.
 - [10] Fengfu Li, Bo Zhang, and Bin Liu. Ternary weight networks. arXiv :1605.04711, 2016.
 - [11] Patrick Nectoux, Rafael Gouriveau, Kamal Medjaher, Emmanuel Ramasso, Bechir Chebel-Morello, Nouredine Zerhouni, and Christophe Varnier. PRONOSTIA : An experimental platform for bearings accelerated degradation tests. In *IEEE International Conference on Prognostics and Health Management (PHM)*, 2012.
 - [12] Mohammad Rastegari, Vicente Ordonez, Joseph Redmon, and Ali Farhadi. XNOR-Net : ImageNet classification using binary convolutional neural networks. In *European Conference on Computer Vision (ECCV)*, 2016.
 - [13] Leonardo Ravaglia, Manuele Rusci, Davide Nadalini, Alessandro Capotondi, Francesco Conti, and Luca Benini. A TinyML platform for on-device continual learning with quantized latent replays. *IEEE Journal on Emerging and Selected Topics in Circuits and Systems*, 11(4) :789–802, 2021.
 - [14] Haoyu Ren, Darko Anicic, and Thomas A. Runkler. TinyOL : TinyML with online-learning on microcontrollers. arXiv :2103.08295, 2021.

- [15] Jonathan Schwarz, Wojciech Czarnecki, Jelena Luketina, Agnieszka Grabska-Barwinska, Yee Whye Teh, Razvan Pascanu, and Raia Hadsell. Progress & compress : A scalable framework for continual learning. In *International Conference on Machine Learning (ICML)*, 2018.

A Balayage de profondeur et de symétrie

La Section 6 ne cite que les points de décision du balayage sub-INT8. Les Tableaux 9 et 10 donnent la grille complète en Δ AUROC, dont se déduisent la frontière ternaire de cette métrique et l’inutilité du zéro-point affine. Le F1 mesuré sur carte des mêmes schémas, qui déplace cette frontière à l’INT4, est donné au Tableau 8.

TABLE 9 – Balayage de profondeur complet : Δ AUROC contre la référence FP32, par profondeur et granularité (*émulé PC bit-exact*). Un Δ négatif est une dégradation.

Profondeur	Monitoring		Pronostia	
	Par tenseur	Par canal	Par tenseur	Par canal
int8	0.0000	0.0002	-0.0009	-0.0009
int6	-0.0000	0.0003	-0.0005	-0.0007
int4	0.0011	-0.0005	-0.0012	-0.0016
int3	0.0026	-0.0026	-0.0021	-0.0021
int2	-0.0028	-0.0069	-0.0464	-0.0086
ternaire	-0.0021	-0.0021	-0.0153	-0.0153
binaire	-0.0117	-0.0117	-0.0275	-0.0275

TABLE 10 – Symétrie de la quantification : Δ AUROC symétrique contre affine à zéro-point (*émulé PC bit-exact*).

Schéma	Monitoring		Pronostia	
	Symétrique	Affine	Symétrique	Affine
int4	-0.0005	-0.0554	-0.0016	-0.0019
int3	-0.0026	-0.0591	-0.0021	-0.0028
int2	-0.0069	-0.0619	-0.0086	-0.0125

B Détail du budget système

Le Tableau 11 décompose la RAM que la Section 7 n’agrège qu’en total : on y lit directement que le `.bss` est *identique* entre FP32 et INT8, l’écart de

total ne venant que du pic de pile. Le Tableau 12 range les trois estimateurs d’énergie côte à côte.

TABLE 11 – Composantes de la RAM, en octets (*mesuré sur carte*). Le total est la somme des trois premières lignes, le pic de pile étant le maximum des deux phases.

Composante	Monitoring		Pronostia	
	FP32	INT8	FP32	INT8
.data	460	460	460	460
.bss	100152	100152	105036	105036
Pic de pile (inférence)	4416	4328	4424	4336
Pic de pile (mise à jour)	4688	4712	4696	4720
Total	105300	105324	110192	110216
Rapport	1.0002		1.0002	

TABLE 12 – Les trois estimateurs d’énergie, côte à côte et jamais fusionnés (*mesuré sur carte*). En μJ par inférence.

Estimateur	FP32	INT8
Régression en cadence	67.65	67.47
Différence contre repos WFI	146.77	145.03
Lot d’inférences par trame	5.08	–

C Registre des mesures manquantes

Le Tableau 13 liste les cellules que l’article laisse explicitement « à mesurer », avec la raison de chacune. Publier ce registre est le pendant de la règle « aucun chiffre inventé » : il rend l’absence vérifiable au lieu de la laisser deviner.

Acronymes

Area Under the ROC Curve (AUROC) Aire sous la courbe ROC : probabilité qu’un échantillon anormal reçoive un score plus élevé qu’un échantillon normal. Indépendante du seuil de décision. 17

Bit Operation (BOP) Coût exprimé au niveau du bit, proportionnel au produit des largeurs des opérandes ; il prédit un facteur 16 entre

TABLE 13 – Registre des cellules non mesurées. Elles portent « à mesurer » dans le texte et ne sont jamais remplacées par une estimation.

Cellule	Raison
energy.estimators.delta_wfi.cellspagne_fp32_energy_ul;pc;update	MAJ CL exige la même mesure avec...
energy.estimators.delta_wfi.cellspagne_int8_energy_ul;pc;update	MAJ CL exige la même mesure avec...
energy.estimators.policy_update.différents monodimensionnels en énergie;pc;update	de always, frozen n'est pas...
energy.estimators.policy_update.différents monodimensionnels en énergie;pc;update	de always, frozen n'est pas...
monitoring.ram.fp32_pc_bss	valeur absente de experiments/exp_S49_ram/summary.json
monitoring.ram.fp32_pc_data	valeur absente de experiments/exp_S49_ram/summary.json
monitoring.ram.fp32_pc_stack_pc;update	valeur absente de experiments/exp_S49_ram/summary.json
monitoring.ram.int8_pc_bss	PC = proxy tracemalloc FP32 ; le gain INT8 (poids ÷4) est analytique — la...
monitoring.ram.int8_pc_data	PC = proxy tracemalloc FP32 ; le gain INT8 (poids ÷4) est analytique — la...
monitoring.ram.int8_pc_stack_P64k;update	PC = proxy tracemalloc FP32 ; le gain INT8 (poids ÷4) est analytique — la...
monitoring.ram.int8_pc_stack_P64k;update	PC = proxy tracemalloc FP32 ; le gain INT8 (poids ÷4) est analytique — la...
monitoring.ram.int8_pc_total	PC = proxy tracemalloc FP32 ; le gain INT8 (poids ÷4) est analytique — la...
monitoring.recovery_board.int8_legacy_log;pc;update	legacy log;pc;update streamée (carte requise)
monitoring.recovery_board.int8_legacy_log;pc;update	legacy log;pc;update streamée (carte requise)
monitoring.recovery_board.q15_chill;pc;update	chill;pc;update non streamée (carte requise)
monitoring.recovery_board.q15_chill;pc;update	chill;pc;update non streamée (carte requise)
pronostia.ram.fp32_pc_bss	valeur absente de experiments/exp_S49_ram/summary.json
pronostia.ram.fp32_pc_data	valeur absente de experiments/exp_S49_ram/summary.json
pronostia.ram.fp32_pc_stack_pc;update	valeur absente de experiments/exp_S49_ram/summary.json
pronostia.ram.int8_pc_bss	PC = proxy tracemalloc FP32 ; le gain INT8 (poids ÷4) est analytique — la...
pronostia.ram.int8_pc_data	PC = proxy tracemalloc FP32 ; le gain INT8 (poids ÷4) est analytique — la...

FP32 et INT8 que la carte ne restitue pas, ce qui illustre les limites d'une métrique analytique. 22

Common Microcontroller Software Interface Standard — Neural Network (CMSIS-NN)

Bibliothèque de noyaux de réseaux de neurones optimisés pour les cœurs Cortex-M, exploitant les instructions vectorielles ; piste directe pour lever le paradoxe de latence constaté ici. 23

Continual Learning (CL) Apprentissage d'une suite de tâches sans accès aux données passées, où le modèle doit intégrer le nouveau sans effacer l'ancien. 2

Core Coupled Memory (CCM) Mémoire rapide directement rattachée au cœur du processeur, non accessible à certains périphériques ; sur la carte cible elle complète la mémoire vive statique principale. 11

Cyclic Redundancy Check (CRC) Somme de contrôle calculée sur chaque trame transmise ; un taux d'erreur nul garantit que les prédictions comparées sont bien celles produites par la carte et non le produit d'une transmission corrompue. 2

Data Watchpoint and Trace (DWT) Unité de débogage du cœur Cortex-M dont le compteur de cycles permet de chronométrer une section de code au cycle près, puis de la convertir en microsecondes à partir de la fréquence d'horloge. Toutes les latences de cet article en sont issues. 10

Elastic Weight Consolidation (EWC) Méthode anti-oubli qui ajoute à la perte une pénalité quadratique freinant la modification des poids jugés importants pour les tâches passées, l'importance étant estimée par l'information de Fisher. Elle ne stocke aucune donnée passée, ce qui la rend compatible avec un microcontrôleur. 2

Floating-Point Operation (FLOP) Opération élémentaire en virgule flottante ; une multiplication-accumulation en compte deux. 21

Floating-Point Unit (FPU) Unité matérielle de calcul en virgule flottante ; sur la carte cible elle rend les opérations FP32 aussi rapides que les opérations entières, ce qui annule le bénéfice de vitesse attendu de la quantification. 4

Hyperdimensional Computing (HDC) Paradigme non neuronal encodant les signaux dans des vecteurs de très haute dimension ; l'apprentissage s'y réduit à des opérations élémentaires (addition, vote majoritaire), sans rétropropagation. 3

- Microcontroller Unit (MCU)** Processeur intégrant sur une même puce cœur, mémoire et périphériques, destiné aux systèmes embarqués autonomes. 2
- Multiply-Accumulate (MAC)** Opération élémentaire multipliant deux valeurs et ajoutant le produit à un accumulateur ; unité de compte du coût d’une couche dense. 10
- Neural Processing Unit (NPU)** Circuit spécialisé dans les opérations de réseaux de neurones, capable d’exécuter des multiplications-accumulations entières massivement en parallèle. Son absence sur la carte cible explique qu’INT8 n’y apporte aucun gain de vitesse. 2
- Post-Training Quantization (PTQ)** Quantification appliquée à un modèle déjà entraîné : simple à mettre en œuvre, mais sujette à des pertes lorsque l’échelle est fixée sans calibration ou que la dynamique des tenseurs est large. C’est le régime dont cet article mesure l’effondrement puis la récupération. 2
- Quantization-Aware Training (QAT)** Quantification simulée à chaque pas d’entraînement par *fake-quant*, de sorte que le modèle apprenne directement sous contrainte de précision réduite ; plus coûteuse à l’entraînement, elle n’apporte ici aucun gain décisif face à une PTQ correctement calibrée. 2
- Random Access Memory (RAM)** Mémoire vive volatile où résident les variables et les tampons du programme ; ressource la plus critique de la cible. 2
- Single Instruction, Multiple Data (SIMD)** Jeu d’instructions appliquant une même opération à plusieurs données en un cycle ; c’est le levier qui manque au chemin entier de ce travail pour convertir le gain de format en gain de temps. 23
- Static Random Access Memory (SRAM)** Mémoire vive statique intégrée au microcontrôleur, rapide mais coûteuse en surface de silicium, donc disponible en très faible quantité. C’est elle qui fixe le budget mémoire de ce travail. 2
- Stochastic Gradient Descent (SGD)** Algorithme d’optimisation mettant à jour les poids dans la direction opposée au gradient, estimé sur un petit lot d’exemples ; c’est la mise à jour exécutée à bord dans le protocole en ligne. 6
- Tiny Machine Learning (TinyML)** Apprentissage automatique sur microcontrôleur, caractérisé par trois contraintes cumulatives : pas de mémoire externe, pas de système d’exploitation, et une mémoire vive inférieure à 256 Ko. 3

Tiny Online Learning (TinyOL) Approche où un réseau préentraîné est gelé en Flash et où seule une couche entraînable légère est mise à jour en RAM, échantillon par échantillon. 3

Universal Asynchronous Receiver-Transmitter (UART) Liaison série asynchrone reliant la carte à l'hôte ; elle achemine les échantillons trame par trame et rapporte les prédictions. 11

Wait For Interrupt (WFI) Instruction mettant le cœur en sommeil jusqu'à la prochaine interruption ; son emploi dans l'attente de la liaison série abaisse le courant de repos et rend exploitable la mesure d'énergie par différence. 11